



Modelowanie zmian zasięgu pokrywy lodowej na biegunie południowym.

Projekt zaliczeniowy z przedmiotu Modelowanie w naukach o Ziemi

Autorki:

Agata Ćwikła, Zofia Gil

Akademia Górniczo Hutnicza w Krakowie
Wydział Geologii, Geofizyki i Ochrony Środowiska

26 Czerwca 2025

1 Cel projektu

Celem projektu było zobrazowanie zmian zasięgu pokrywy lodowej na biegunie południowym w latach 1978 – 2009 oraz stworzenie modelu matematycznego zasięgu lodu zależnego od czasu, odwzorowującego dane rzeczywiste.

2 Dane źródłowe

Projekt został wykonany na podstawie danych z pliku `daily_ice_edge.csv` który zawiera informacje o zasięgu lodu morskiego wokół Antarktydy dla wszystkich długości geograficznych w formacie długość-szerokość dla wybranych dni na przestrzeni lat 1978 - 2009.

3 Metodyka wykonania projektu

Projekt został wykonany głównie w języku programowania R w środowisku *jupyter-notebook*. Do stworzenia animacji dodatkowo wykorzystano język programowania Python. Pierwszym etapem projektu było wyznaczenie minimalnego i maksymalnego zasięgu lodu dla całego badanego okresu. W tym celu po zaimportowaniu danych do środowiska wykonawczego i ich wstępnej obróbce (usunięciu pierwszej kolumny zawierającej daty pomiarów) oraz wygenerowaniu listy azymutów (w stopniach oraz radianach),

```
szerokosci_mat <- dane[, -1]
azymuty <- 0:360
azymuty_rad <- azymuty * pi / 180
```

użyto funkcji *apply()* zastosowanej do kolumn. Otrzymano wówczas wektor wartości numerycznych zawierający najmniejsze/największe wartości zasięgu lodu w każdym kierunku.

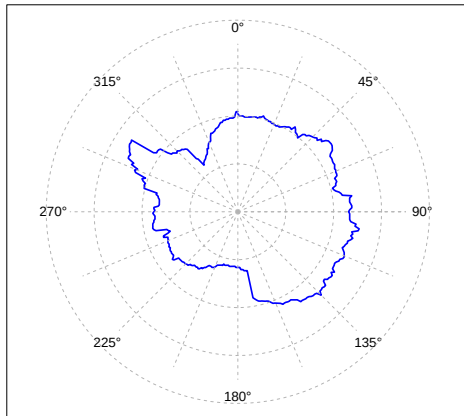
```
min_szerokosci <- apply(szerokosci_mat, 2, min, na.rm = TRUE)
max_szerokosci <- apply(szerokosci_mat, 2, max, na.rm = TRUE)
```

Następnie za pomocą prostych przekształceń trygonometrycznych przeliczono otrzymane wartości tak, aby można było je prawidłowo zwizualizować na wykresie.

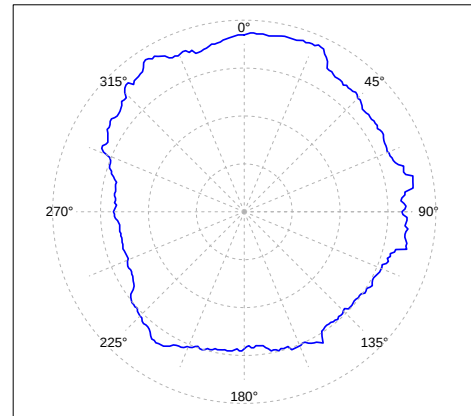
```
r_min <- 90 - abs(min_szerokosci) # odległość od bieguna S
y_min <- r_min * cos(azymuty_rad)
x_min <- r_min * sin(azymuty_rad)
#analogicznie dla max_szerokosci
```

Wykresy wykonano za pomocą funkcji *plot()* z pakietu bazowego R. Poniżej wizualizacja najmniejszego oraz największego zasięgu lodu dla całego okresu.

Minimalny zasięg lodu na biegunie południowym (pełne dane)

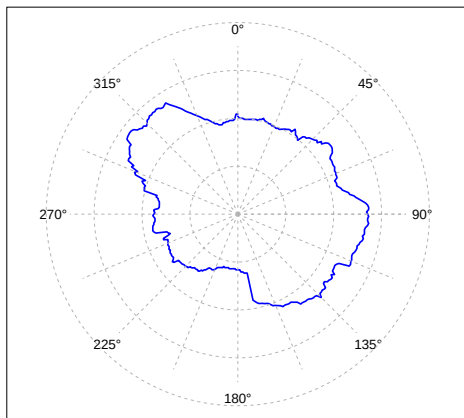


Maksymalny zasięg lodu na biegunie południowym (pełne dane)

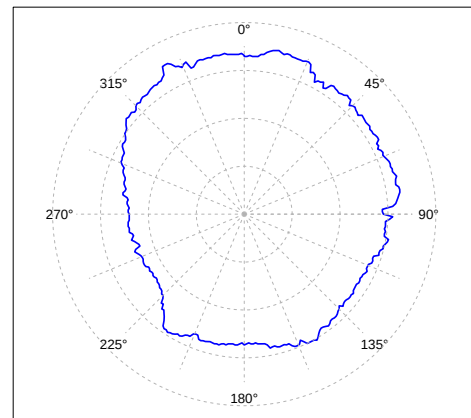


W ramach ciekawoski wyznaczono również konkretne dni, podczas których pokrywa lodowa była odpowiednio najmniejsza i największa.

Dzien z najmniejsza pokrywa lodowa: 2009-05-10

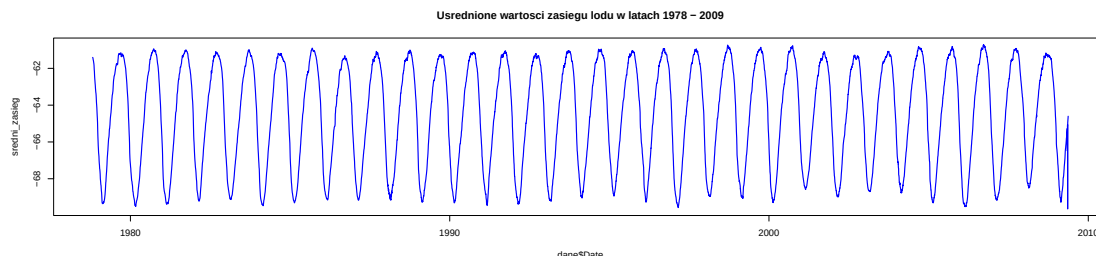


Dzien z największa pokrywa lodowa: 2006-09-21



4 Model matematyczny

Przed przystąpieniem do wykonania modelu sprawdzono zachowanie analizowanych danych, przedstawione na poniższym wykresie.



Wyraźnie widać, że dane mają rozkład sinusoidalny, dlatego do stworzenia modelu odwzorowującego dane rzeczywiste użyto funkcji sinus oraz, w celu zwiększenia dokładności, funkcji cosinus. Dzięki temu, że zastosowano obie te funkcje trygonometryczne, model jest bardziej elastyczny w dopasowywaniu się do danych i nie ma na stałe ustalonego miejsca, w którym znajduje się minimum czy maximum funkcji.

W celu wygenerowania parametrów pasujących do modelu użyto funkcji *lm()*, która umożliwia dopasowanie modelu liniowego z uwzględnieniem okresowości. W generowanym modelu zmienną zależną jest zasięg lodu morskiego i to on został wymodelowany za pomocą funkcji trygonometrycznych z uwzględnieniem czasu - zmiennej niezależnej oraz sezonowości danych. Aby dopasować funkcje sinus i cosinus do zaobserwowanej okresowości danych, wyskalowano ich argumenty tak, żeby po ok 365 jednostkach czasu (w tym przypadku dni) wartość funkcji się powtarzała. Należy jednak pamiętać, że nie jest to idealne dopasowanie, ponieważ dane nie są kompletne we wszystkie dni z analizowanego okresu.

Następnie za pomocą funkcji *predict()* obliczono przewidywane wartości z dopasowanego wcześniej modelu fit dla całego przedziału czasowego *t_num*. Otrzymane wartości zapisano do ramki danych przechowującej wymodelowane dane dla każdego azymutu.

Zarówno fragment obliczający parametry modelu, jak również część obliczająca wymodelowane dane, znajdują się w pętli, która iteruje po wszystkich wartościach azymutów. Upewniono się również, aby nieprawidłowe dane lub ich brak nie powodowały konfliktów w działaniu funkcji.

```
#n_kat - liczba azymutów
#t_num - wektor numeryczny zawierający liczbę dni od pierwszej daty
#dane_i_mod - tymczasowa ramka danych w pętli zawierająca kolumnę t_num oraz
#kolumnę y - wartość zasięgu lodu w danym azymucie
#365 - ustalona wartość sezonowości danych

for (j in 1:n_kat) {
  y <- szerokosci_mat[, j]

  dane_i_mod <- data.frame(t_num = t_num, y = y)
  dane_i_mod <- na.omit(dane_i_mod)

  if (nrow(dane_i_mod) > 10) { # minimalna liczba punktów do dopasowania
    fit <- lm(y ~
      sin(2 * pi * t_num / 365) +
      cos(2 * pi * t_num / 365),
      data = dane_i_mod)

    pred_mod <- predict(fit, newdata = data.frame(t_num = t_num))

    model_pred[, j] <- pred_mod
  } else {
```

```

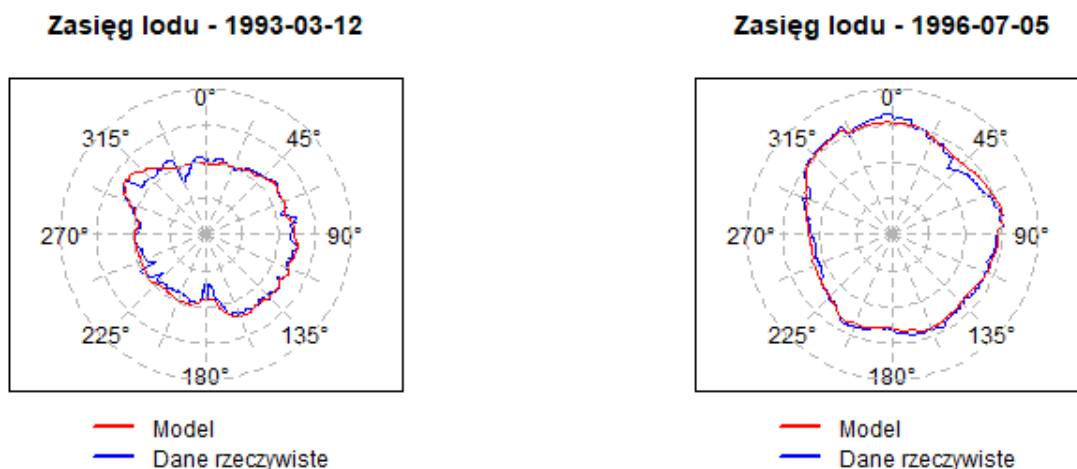
    model_pred[, j] <- NA
  }
}

```

5 Animacja

Podczas generowania animacji największą przeszkodą okazała się niewystarczająca ilość pamięci podręcznej. W celu poradzenia sobie z problemem wypróbowano wiele wariantów postępowania takich jak: zwiększanie limitów pamięci operacyjnej, generowanie animacji w częściach i późniejsze ich łączenie, zapisywanie generowanych wykresów do plików i odczytywanie je przez m.in funkcje z pakietów *magick* oraz *animation*. Niestety żadna z tych metod nie okazała się wystarczająco dobra. Dlatego do stworzenia animacji przedstawiającej zarówno rzeczywiste jak i wymodelowane dane użyto również języka Python. Wcześniej wygenerowane w środowisku R i zapisane do folderu wykresy w formacie *png* odczytano za pomocą skryptu, który przekazał je do programu *FFmpeg*, któremu stworzenie animacji zajęło niewspółmiernie krótki czas (ok 2 minuty, podczas gdy niektóre z wcześniejszych prób generowania animacji trwały nawet 5-6 godzin).

Poniżej przedstawiono przykładowe kadry z animacji.



Sposób dla wszystkich danych

W celu wykorzystania wszystkich danych jednocześnie, należałoby rozszerzyć model z zależności zasięgu lodu od czasu do zasięgu lodu w zależności od czasu oraz długości geograficznej (w przypadku tego projektu są to wartości azymutów). Dotychczasowy model był tworzony dla każdego azymutu/długości z osobna. W celu stworzenia modelu uwzględniającego wszystkie azymuty naraz za pomocą modelu liniowego, należałoby przekształcić oryginalną ramkę danych na ramkę danych zawierającą 3 kolumny: zasięg lodu (tak jak w danych oryginalnych), azymuty, kolumnę z liczbą dni od początku pomiarów. W sumie ramka danych zawierałaby (liczba dni) razy (360 azymutów) wierszy. Wtedy można by rozbudować wcześniejszy model o wartości azymutów i zamienić iteracyjne wyliczanie modelu dla każdego kąta na podejście globalne. Model mógłby wyglądać w taki sposób:

```

y ~
sin(2 * pi * t_num / 365) +
cos(2 * pi * t_num / 365) +
#do tego momentu tak samo
#dodanie zmienności w zależności od azymutu
sin(azymut) +
cos(azymut) +
#opcjonalne dodanie zależności pomiędzy azymutem a czasem
sin(azymut) * t_num +
cos(azymut) * t_num

```

Podsumowanie i wnioski

Analizując otrzymaną animację można stwierdzić, że model zdołał dopasować się do danych, szybko reaguje na ich zmienność i z powodzeniem może być wykorzystywany do przewidywania zachowania się pokrywy lodowej w przyszłości. Model cechuje się większą stabilnością i przewidywalnością, w odróżnieniu od danych rzeczywistych, w których występują błędy i wartości odstające. Na podstawie wykonanego projektu można również stwierdzić, że warto wykorzystywać różne środowiska i języki programowania w zależności od charakteru przetwarzanych danych i potrzeb projektowych.